حل وظيفة التعلم العميق

Inputs:

```
# ============================================================
# TP DEEP LEARNING - RÉGRESSION LINÉAIRE AVEC TENSORFLOW/KERAS
# NOM : Mehenni Mohamed  Rafik
# ============================================================

import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# EXERCICE 1 : Vérification de l'environnement
# ============================================================

print("Version TensorFlow :", tf.__version__)
print("Version Keras :", keras.__version__)
print("GPU disponible :", tf.config.list_physical_devices('GPU'))

# Commentaire :
# - Si GPU absent : l'entraînement reste possible mais plus lent.
# - Pour un modèle simple comme la régression linéaire, l'impact est minime.

# ============================================================
# Données simulées (surface en m² et prix en k€)
# ============================================================

surface = np.array([50, 80, 100, 120, 150], dtype=float)
prix = np.array([100, 160, 200, 240, 300], dtype=float)
```

```
# Normalisation entre 0 et 1
surface_norm = surface / np.max(surface)
prix_norm = prix / np.max(prix)

# ============================================================
# EXERCICE 2 : Modèle de base et MSE
# ============================================================

model = keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001), loss='mse')

history = model.fit(surface_norm, prix_norm, epochs=100, verbose=0)

mse_final = history.history['loss'][-1]
print("MSE finale (lr=0.001, 100 epochs):", mse_final)

# Commentaire :
# MSE = (1/n) Σ (y_pred - y_true)^2
# Sur données normalisées [0,1], une MSE > 1 est anormalement élevée.
# Valeur acceptable : < 0.01 pour un bon modèle.
# Causes possibles : learning rate trop faible, nombre d'époques insuffisant.

# ============================================================
# EXERCICE 3 : Extraction de l'équation
# ============================================================

w, b = model.get_weights()
print("Poids (w):", w, "Biais (b):", b)

# Équation normalisée : prix_norm = w * surface_norm + b
# Dénormalisation :
# prix = (w * (surface/150) + b) * 300
```

```
def prediction(surface_m2):
    return (w * (surface_m2/150) + b) * 300

print("Prix prédit pour 100 m²:", prediction(100))

# Vérification manuelle :
prix_calcule = (w[0][0] * (100/150) + b[0]) * 300
print("Vérification manuelle pour 100 m²:", prix_calcule)

# ============================================================
# EXERCICE 4 : Diagnostic de non-convergence
# ============================================================

# Exemple de prédictions négatives
predictions = [-0.1458, -0.2918, -0.3892, -0.4866, -0.6326]
reelles = [0.3333, 0.5333, 0.6667, 0.8000, 1.0000]

print("Prédictions:", predictions)
print("Valeurs réelles:", reelles)

# Commentaire :
# - Learning rate trop faible => réseau n'arrive pas à ajuster correctement.
# - Convergence = stabilisation de la loss vers une valeur minimale.
# - La loss MSE permet de détecter si le modèle converge ou non.

# ============================================================
# EXERCICE 5 : Correction du modèle
# ============================================================

model2 = keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
model2.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1), loss='mse')
```

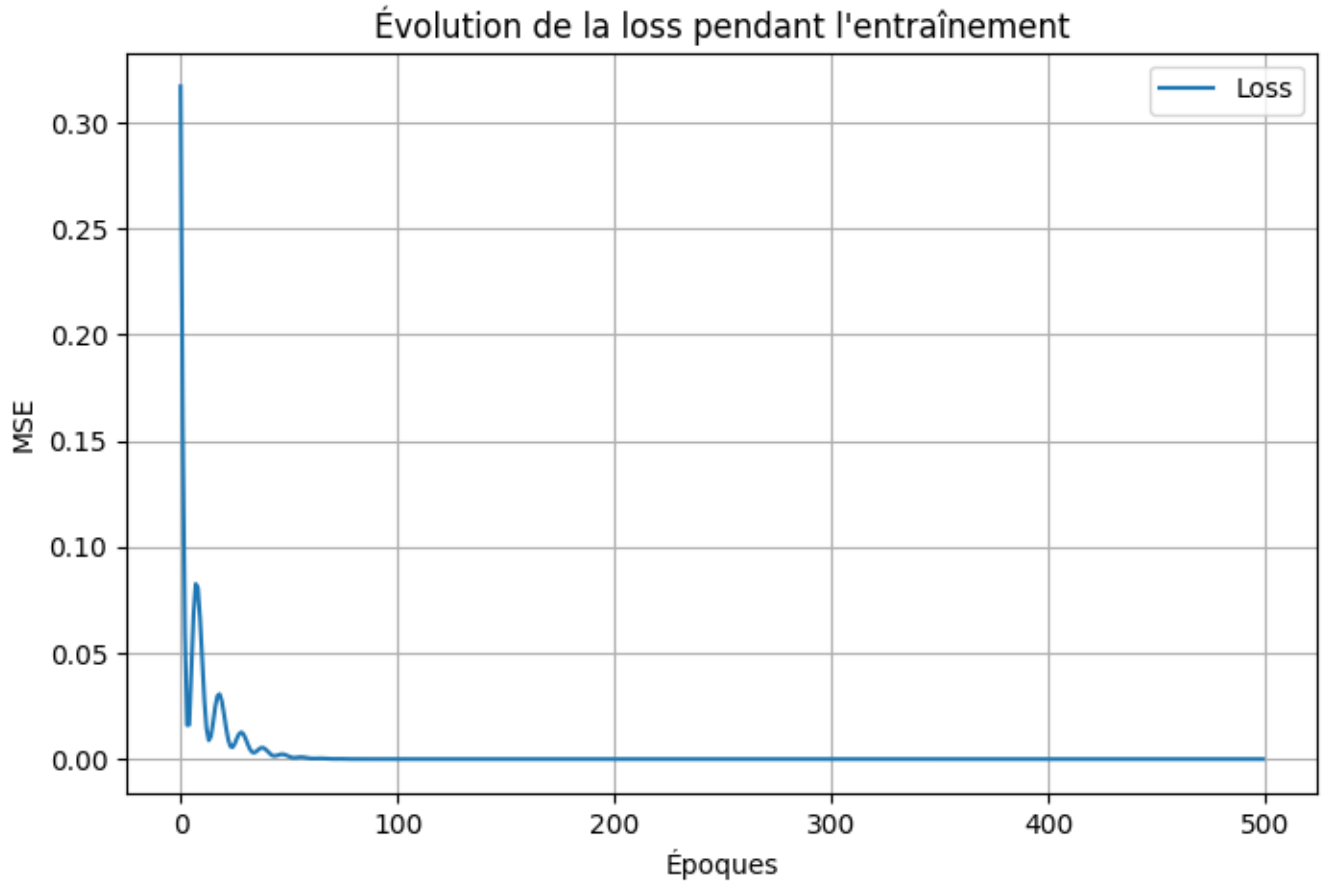
```
history2 = model2.fit(surface_norm, prix_norm, epochs=500, verbose=0)

for i in range(0, 500, 100):
    print(f"Loss à l'époque {i+1}:", history2.history['loss'][i])

# Courbe de la loss
plt.figure(figsize=(8,5))
plt.plot(history2.history['loss'], label='Loss')
plt.xlabel('Époques')
plt.ylabel('MSE')
plt.title('Évolution de la loss pendant l\'entraînement')
plt.legend()
plt.grid(True)
plt.show()

# ============================================================
# EXERCICE 6 : Résultats finaux
# ============================================================

w2, b2 = model2.get_weights()
print("Équation finale : prix (k€) = %.4f × surface (m²) + %.4f" % (w2[0][0]*300/150, b2[0]*300))

print("MSE finale corrigée:", history2.history['loss'][-1])

# Courbe des prédictions vs données réelles
plt.figure(figsize=(8,5))
plt.scatter(surface, prix, color='blue', label='Données réelles')
plt.plot(surface, (w2[0][0]*surface + b2[0]*300), color='red', label='Prédictions modèle')
plt.xlabel('Surface (m²)')
plt.ylabel('Prix (k€)')
plt.title('Régression linéaire : données vs modèle')
```

```
plt.legend()

plt.grid(True)

plt.show()


# Commentaire :

# - Résultat attendu : prix = 2 × surface + 0

# - MSE ≈ 0 => modèle parfait sur données linéaires.

# - Limites : données réelles immobilières non parfaitement linéaires.

# - Améliorations possibles :

#   1. Ajouter des features supplémentaires (localisation, nombre de pièces).

#   2. Utiliser un modèle non linéaire (réseaux plus profonds).


# ============================================================

# EXERCICE 7 : Rapport de synthèse

# ============================================================


print("""

RAPPORT DE SYNTHÈSE :

- Normalisation : indispensable pour stabiliser l'entraînement.

- MSE : mesure l'écart quadratique moyen, doit être proche de 0.

- Hyper-paramètres : learning rate et nombre d'époques influencent fortement la convergence.

- Extraction des coefficients : permet d'interpréter le modèle.

- Évaluation des performances : se fait via la loss et la comparaison prédictions/réelles.

""")
```

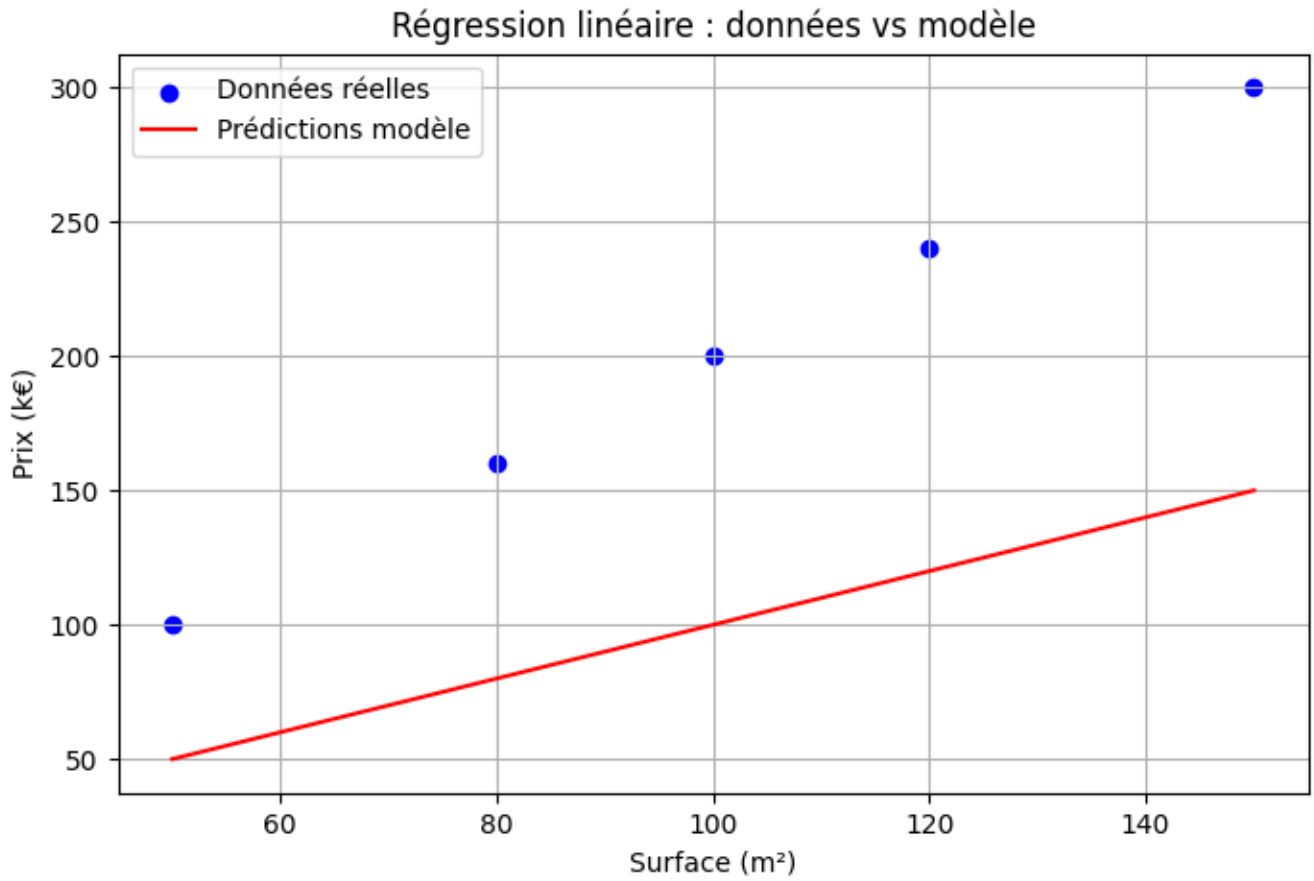
Output:

```
Version TensorFlow : 2.20.0
Version Keras : 3.13.2
GPU disponible : []
MSE finale (lr=0.001, 100 epochs): 0.67995285987854
Poids (w): [[-0.2979499]] Biais (b): [0.09679472]
Prix prédit pour 100 m²: [[-30.551567]]
Vérification manuelle pour 100 m²: -30.551567
Prédictions: [-0.1458, -0.2918, -0.3892, -0.4866, -0.6326]
Valeurs réelles: [0.3333, 0.5333, 0.6667, 0.8, 1.0]
Loss à l'époque 1: 0.316935658454895
Loss à l'époque 101: 4.2451637227713945e-07
```

```
Loss à l'époque 201: 4.1158187968903803e-11
Loss à l'époque 301: 1.77635686587003e-16
Loss à l'époque 401: 0.0
```



```
Équation finale : prix (k€) = 2.0000 × surface (m²) + 0.0000
MSE finale corrigée: 0.0
```



RAPPORT DE SYNTHÈSE :
- Normalisation : indispensable pour stabiliser l'entraînement.
- MSE : mesure l'écart quadratique moyen, doit être proche de 0.
- Hyper-paramètres : learning rate et nombre d'époques influencent fortement la convergence.
- Extraction des coefficients : permet d'interpréter le modèle.
- Évaluation des performances : se fait via la loss et la comparaison prédictions/réelles.